

- The **ADT Matrix** is a container that represents a two-dimensional array.
- Each element has a unique position, determined by two indexes: its line and column.
- The domain of the ADT Matrix: $\mathcal{MAT} = \{mat \mid mat \text{ is a matrix with elements of the type TElem}\}$
- What operations should we have for a Matrix?

- **init(mat, nrL, nrC)**
 - **descr:** creates a new matrix with a given number of lines and columns
 - **pre:** $nrL \in N^*$ and $nrC \in N^*$
 - **post:** $mat \in \mathcal{MAT}$, mat is a matrix with nrL lines and nrC columns
 - **throws:** an exception if nrL or nrC is negative or zero

- **nrLines(mat)**
 - **descr:** returns the number of lines of the matrix
 - **pre:** $mat \in \mathcal{MAT}$
 - **post:** $nrLines \leftarrow$ returns the number of lines from mat

ADT Matrix - Interface III

- **nrCols(mat)**
 - **descr:** returns the number of columns of the matrix
 - **pre:** $mat \in \mathcal{MAT}$
 - **post:** $nrCols \leftarrow$ returns the number of columns from mat

- `element(mat, i, j)`
 - **descr:** returns the element from a given position from the matrix (assume 1-based indexing)
 - **pre:** $mat \in \mathcal{MAT}, 1 \leq i \leq nrLines, 1 \leq j \leq nrColumns$
 - **post:** $element \leftarrow$ the element from line i and column j
 - **throws:** an exception if the position (i, j) is not valid (less than 1 or greater than $nrLines/nrColumns$)

- **modify**(mat, i, j, val)
 - **descr:** sets the element from a given position to a given value (assume 1-based indexing)
 - **pre:** $mat \in \mathcal{MAT}$, $1 \leq i \leq nrLines$, $1 \leq j \leq nrColumns$, $val \in TElem$
 - **post:** the value from position (i, j) is set to val . $modify \leftarrow$ the old value from position (i, j)
 - **throws:** an exception if position (i, j) is not valid (less than 1 or greater than nrLine/nrColumns)

- Other possible operations:
 - get the (first) position of a given element
 - create an iterator that goes through the elements by columns
 - create an iterator the goes through the elements by lines
 - etc.

ADT Matrix - representation

- Usually a sequential representation is used for a Matrix (we memorize all the lines one after the other in a consecutive memory block).
- If this sequential representation is used, for a matrix with N lines and M columns, the element from position (i, j) can be found at the memory address:
address of element from position $(i, j) = \text{address of the matrix} + (i * M + j) * \text{size of an element}$
- The above formula works for 0-based indexing, but can be adapted to 1-based indexing as well.

ADT Matrix - representation

Size of int: 4

Address of matrix (5 rows, 8 cols): 6224024

Address of element 0, 0: 6224024

Address of element 2, 4: 6224104

Address of element 2, 5: 6224108

Address of element 2, 6: 6224112

Address of element 2, 7: 6224116

Address of element 3, 0: 6224120

Address of element 3, 4: 6224136

Address of element 4, 7: 6224180

ADT Matrix - representation

- In the Minesweeper game example above we have a matrix with 480 elements ($16 * 30$) but only 99 bombs.
- If the Matrix contains many values of 0 (or 0_{TElem}), we have a *sparse matrix*, where it is more (space) efficient to memorize only the elements that are different from 0.

Sparse Matrix Example

0	33	0	100	1	0	0	9
2	0	2	0	2	0	7	0
0	4	0	0	3	0	0	0
17	0	0	10	0	16	0	7
0	0	0	0	0	0	0	0
0	1	0	13	0	8	0	29

- Number of lines: 6
- Number of columns: 8
- from 48 positions, 30 values are equal to 0.

- We can memorize (line, column, value) triples, where value is different from 0 (or 0_{TElem}). For efficiency, we memorize the elements sorted by the (line, column) pairs (if the lines are different we order by line, if they are equal we order by column) - R1.
- If a (line, column) pair is not in the representation, we know that its value is 0 (or 0_{TElem}):
- Triples can be stored in a dynamic array or other data structures (that will be discussed later):

Sparse Matrix - R1 example

0	33	0	100	1	0	0	9
2	0	2	0	2	0	7	0
0	4	0	0	3	0	0	0
17	0	0	10	0	16	0	7
0	0	0	0	0	0	0	0
0	1	0	13	0	8	0	29

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Line	1	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	5	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	1	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

Sparse Matrix - R1 example

- In an ADT Matrix, there is no operation to add an element or to remove an element. In the interface we only have the *modify* operation which changes a value from a position. If we represent the matrix as a sparse matrix, the *modify* operation might add or remove an element to/from the underlying data structure. But the operation from the interface is still called *modify*.

- When we have a Sparse Matrix (i.e., we keep only the values different from 0), for the modify operation we have four different cases, based on the value of the element currently at the given position (let's call it *current_value*) and the new value that we want to put on that position (let's call it *new_value*).
 - $current_value = 0$ and $new_value = 0 \Rightarrow$ do nothing
 - $current_value = 0$ and $new_value \neq 0 \Rightarrow$ insert in the data structure
 - $current_value \neq 0$ and $new_value = 0 \Rightarrow$ remove from the data structure
 - $current_value \neq 0$ and $new_value \neq 0 \Rightarrow$ just change the value in the data structure

Sparse Matrix - R1 example

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Line	1	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	5	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	1	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

- Modify the value from position (1, 5) to 0

Sparse Matrix - R1 example

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Line	1	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	5	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	1	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

- Modify the value from position (1, 5) to 0

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Line	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

- Modify the value from position (3, 3) to 19

Sparse Matrix - R1 example

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Line	1	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	5	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	1	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

- Modify the value from position (1, 5) to 0

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Line	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

- Modify the value from position (3, 3) to 19

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Line	1	1	1	2	2	2	2	3	3	3	4	4	4	4	6	6	6	6
Col	2	4	8	1	3	5	7	2	3	5	1	4	6	8	2	4	6	8
Value	33	100	9	2	2	2	7	4	19	3	17	10	16	7	1	13	8	29

- We can see that in the previous representation there are many consecutive elements which have the same value in the line array. The array containing this information could be compressed, in the following way:
 - Keep the *Col* and *Value* arrays as in the previous representation.
 - For the lines, have an array of number of lines + 1 element, in which at position i we have the position from the *Col* array where the sequence of elements from line i begins.
 - Thus, elements from line i are in the *Col* and *Value* arrays between the positions $[Line[i], Line[i+1])$.
- This is called **compressed sparse line representation**.
- **Obs:** In order for this representation to work, in the *Col* and *Value* arrays the elements have to be stored by rows (first elements of the first row, then elements of second row, etc.)

Sparse Matrix - R2 example

Diagram illustrating the mapping of lines to columns in a 2D array:

Col	2	4	5	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	1	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

The 'Lines' array (1, 5, 9, 11, 15, 15, 19) is mapped to the columns of the 2D array. Red arrows indicate the mapping: Line 1 to Col 1, Line 5 to Col 5, Line 9 to Col 9, Line 11 to Col 11, Line 15 to Col 13, and Line 19 to Col 15.

Sparse Matrix - R2 example

- Modify the value from position (1, 5) to 0

Sparse Matrix - R2 example

- Modify the value from position (1, 5) to 0
 - First we look for element on position (1,5).
 - Elements from line 1 are between positions 1 and 4 (inclusive)
 - Since we have there an item with column 5, we found our element
 - Setting to 0, means removing from *Col* and *Value* array.
 - In *Lines* array just the values change, not the size of the array.

	1	2	3	4	5	6	7	
Lines	1	4	8	10	14	14	18	

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Col	2	4	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

Sparse Matrix - R2 example

- Modify the value from position (3, 3) to 19

Sparse Matrix - R2 example

- Modify the value from position (3, 3) to 19
 - First we look for element on position (3,3)
 - Elements from line 3 are between positions 8 and 9 (inclusive)
 - Since we have no column 3 there, at this position currently the value is 0. To set it to 19 we need to insert a new element in the *Col* and *Value* array.
 - In *Lines* array just the values change, not the size of the array

Diagram illustrating the mapping of lines to columns in the 8x8 grid:

	1	2	3	4	5	6	7
Lines	1	4	8	11	15	15	19

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Col	2	4	8	1	3	5	7	2	3	5	1	4	6	8	2	4	6	8
Value	33	100	9	2	2	2	7	4	19	3	17	10	16	7	1	13	8	29

Red arrows indicate the mapping from the 'Lines' row to the 'Col' row:

- Line 1 maps to Column 1
- Line 4 maps to Column 4
- Line 8 maps to Column 8
- Line 11 maps to Column 11
- Line 15 maps to Column 15
- Line 15 maps to Column 14
- Line 19 maps to Column 15